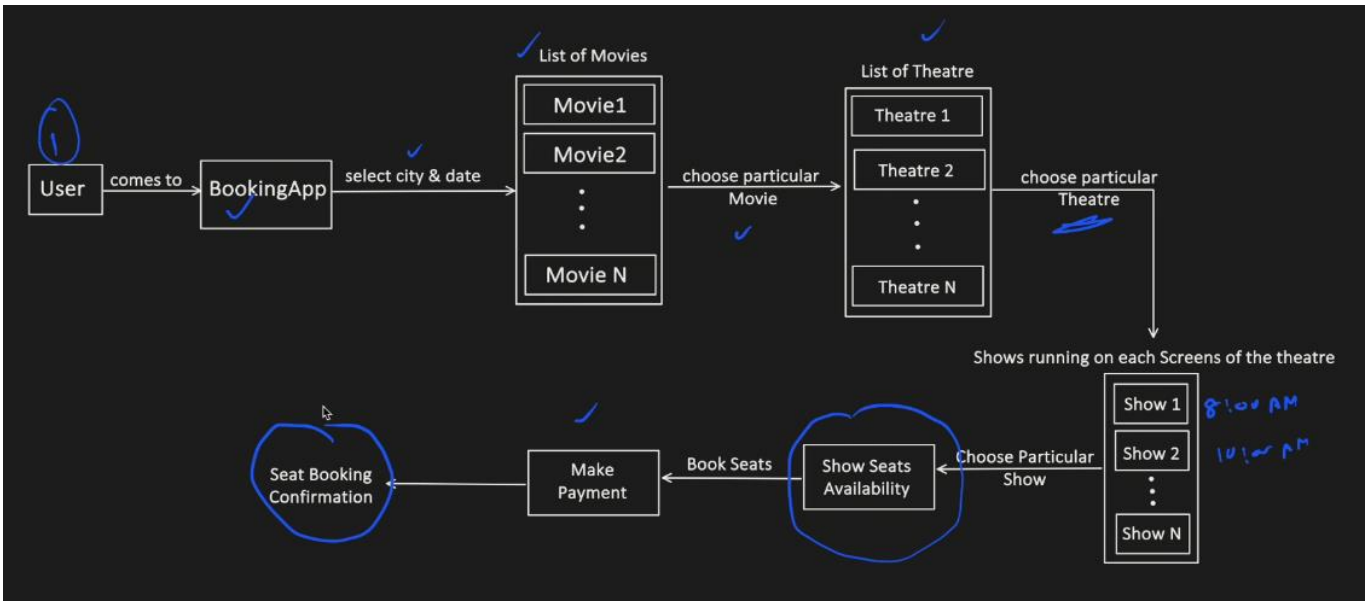
15. LLD of BookMyShow | Design Movie Ticket Booking App

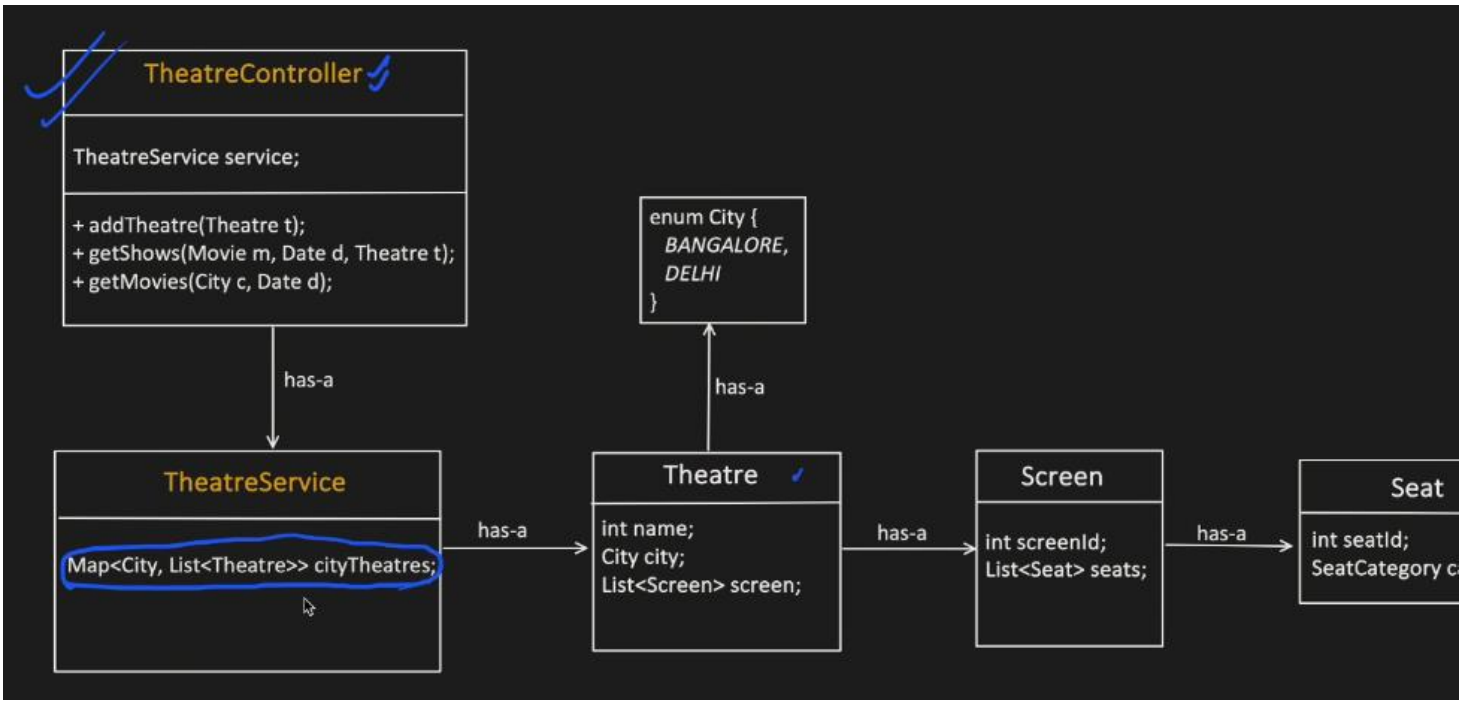
GIT :

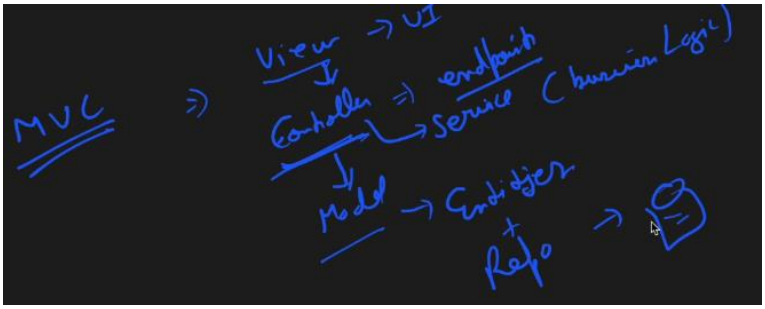
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/interviewquestions/book_my_show?ref_type=heads

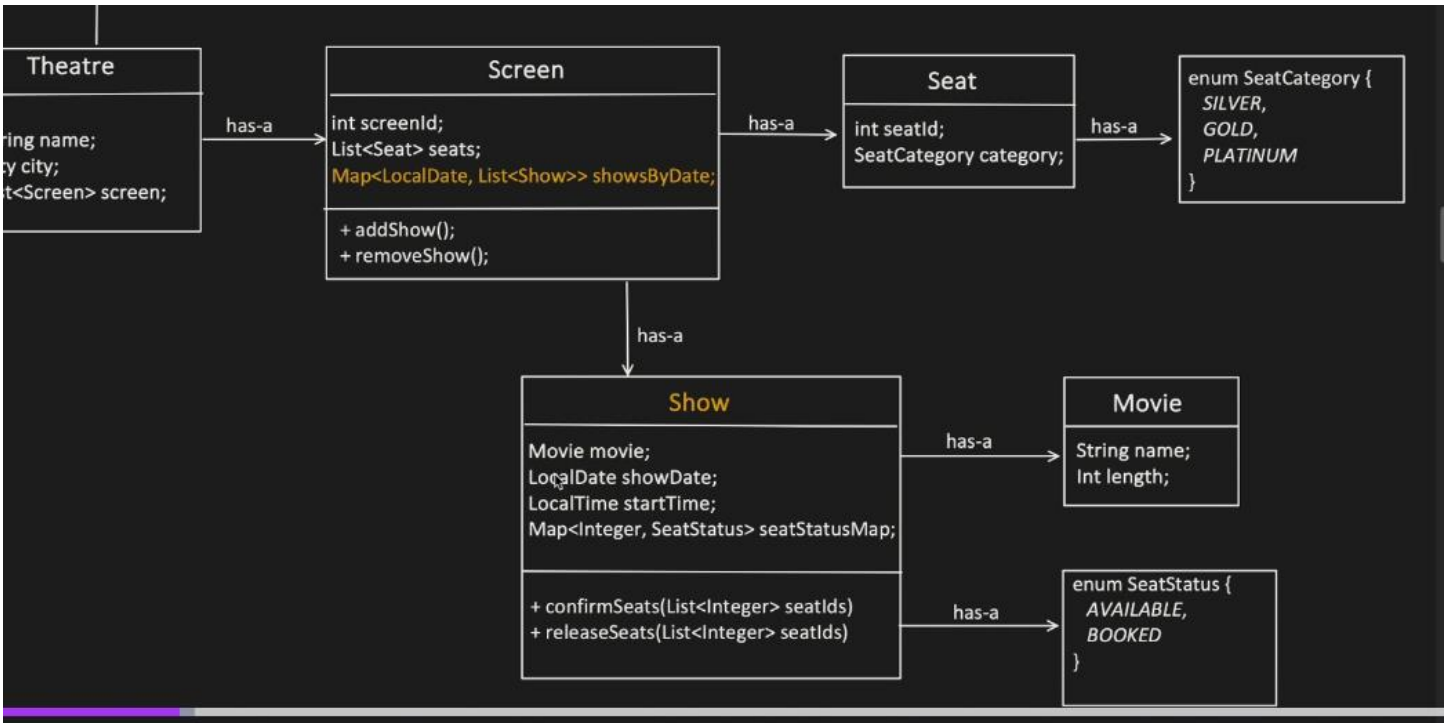
PDF:

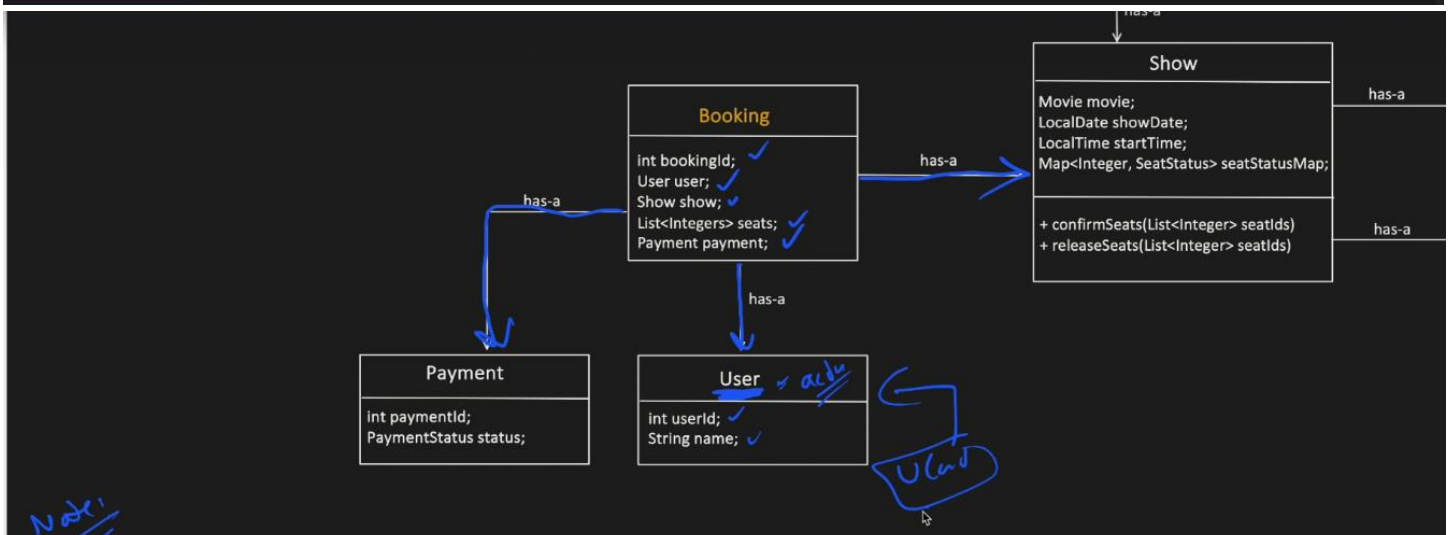
https://drive.google.com/file/d/1XbWo8m1hOOdKGww0UO1x3zIskdQjqf2i/view





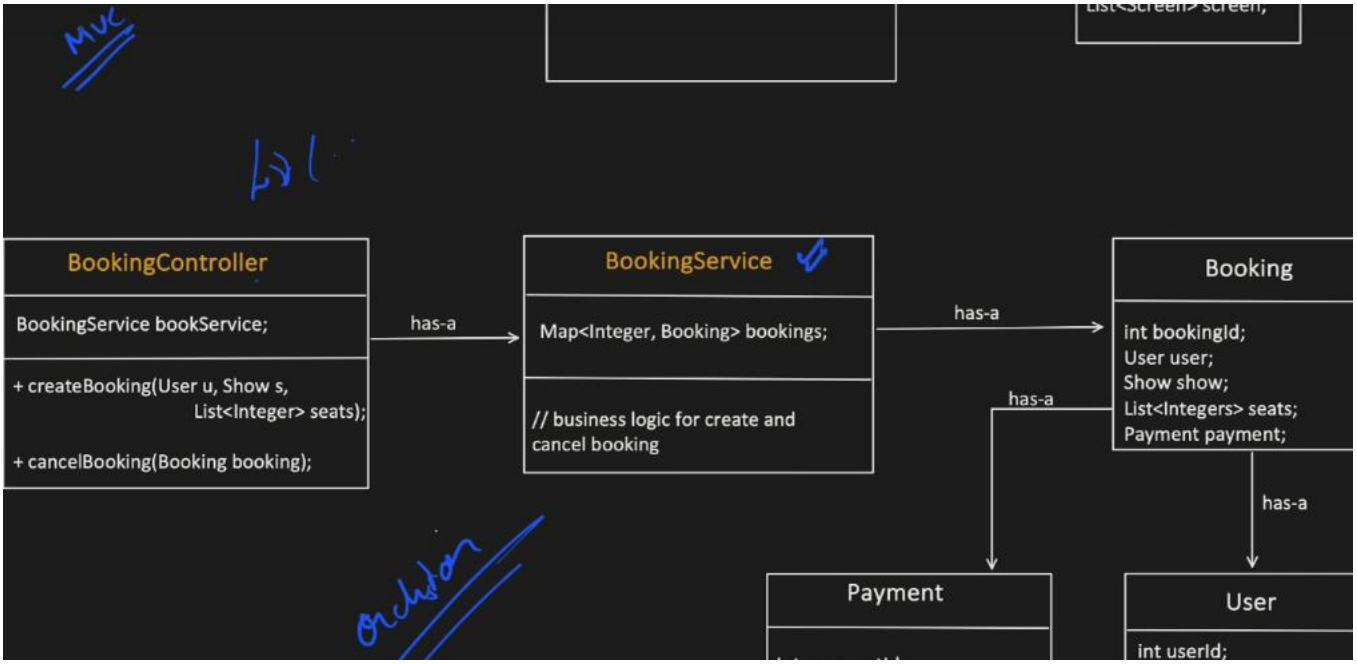



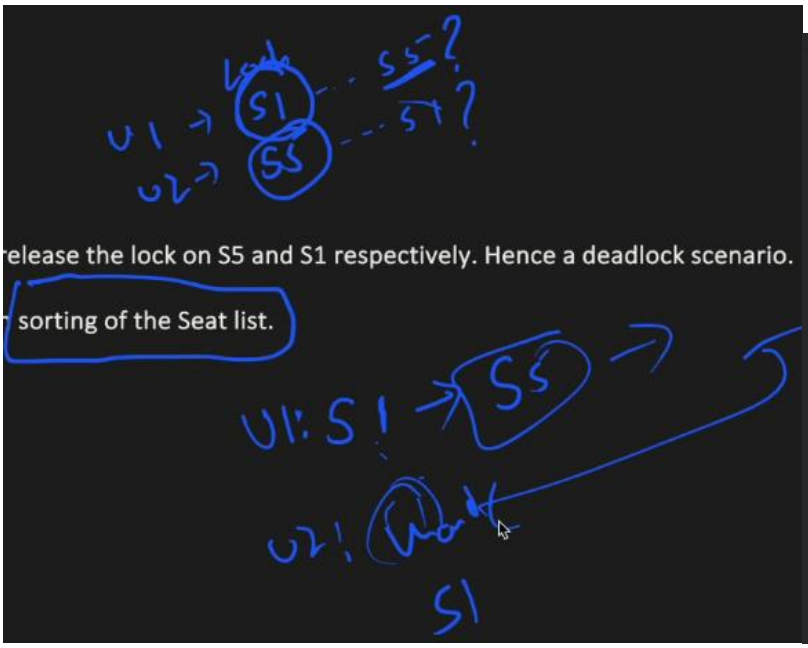

Note:

- if Interviewer says, we need to manage the User lifecycle too like add, remove ,delete. Then we can have its own controller like UserController. But here in booking functionality, we are not creating or managing user, so I am skipping exposing the endpoints for managing the lifecycle of User.

- Similarly, I am not creating PaymentController, PaymentService layer for now, but if interviewer want we can add that too.

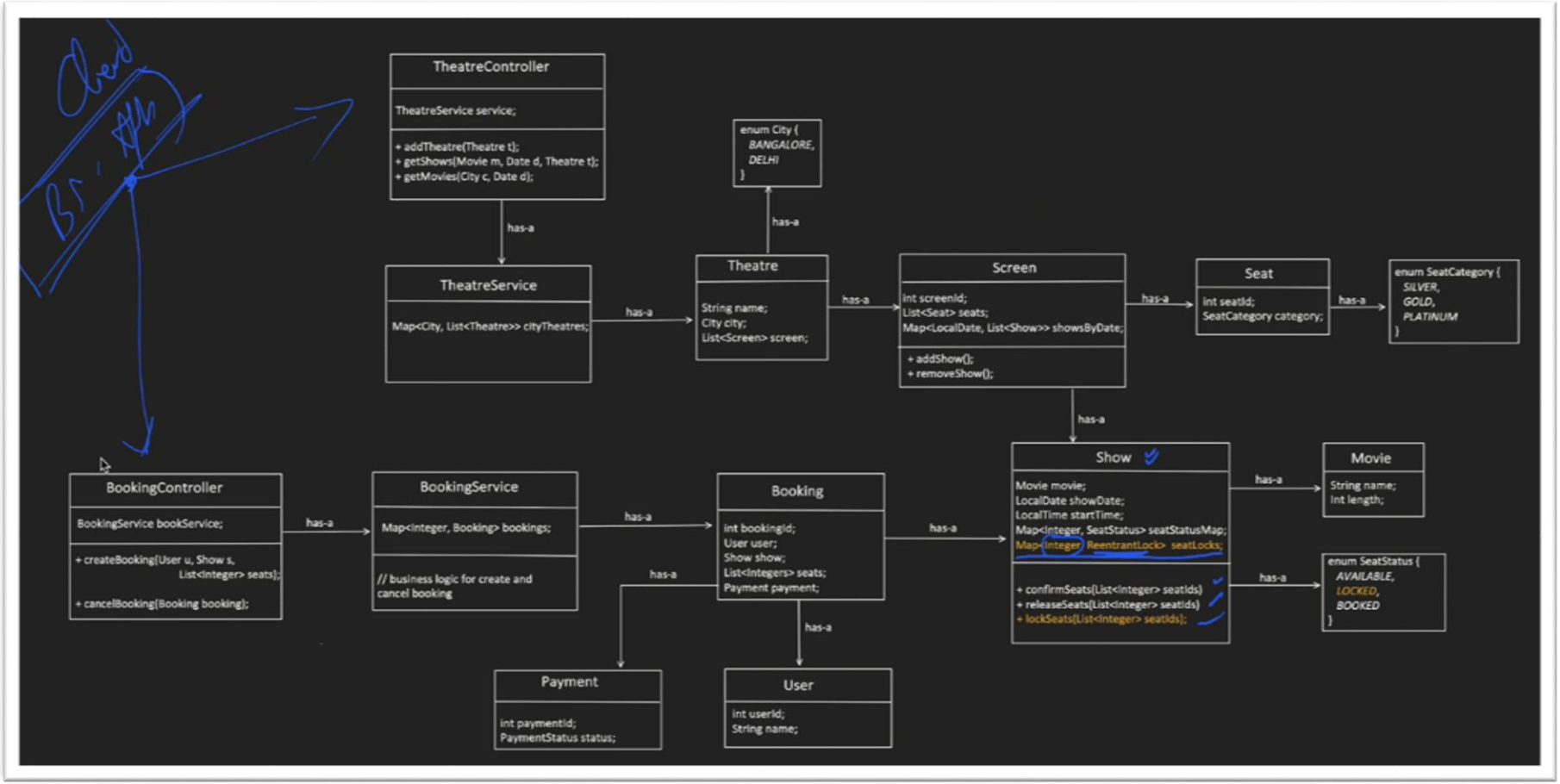



The **Model-View-Controller (MVC)** is an architectural design pattern that separates an application into three interconnected components to isolate business logic from the user interface, improving modularity, maintainability, and scalability. GeeksforGeeks +1




```
/Users/shrayanshjain/Library/Java/JavaVirtualMachin
User logged in: Shrayansh
Selected City: BANGALORE
Selected Date: 2026-01-12
Movies available:
 - BAAHUBALI
 - AVENGERS
Selected Movie: BAAHUBALI
Theatres available:
 - INOX
Selected Theatre: INOX
Shows available:
 - 08:00
 - 15:00
Selected Show Time: 08:00
Selected Seats: [1, 2, 3]
BOOKING SUCCESSFUL
Booking ID: b731fdc6-5745-4026-ab34-dc72d22058e3

Process finished with exit code 0
```


TheatreController
TheatreService service;
+ addTheatre(Theatre t);
+ getShows(Movie m, Date d, Theatre t);
+ getMovies(City c, Date d);
has-a
TheatreService
Map<City, List<Theatre>> cityTheatres;
has-a
enum City {
BANGALORE,
DELHI
}
has-a
Theatre
String name;
City city;
List<Screen> screen;
has-a
Screen
int screenId;
List<Seat> seats;
Map<LocalDate, List<Show>> showsByDate;
+ addShow();
+ removeShow();
has-a
Seat
int seatId;
SeatCategory category;
has-a
enum SeatCategory {
SILVER,
GOLD,
PLATINUM
}
has-a
Show
Movie movie;
LocalDate showDate;
LocalTime startTime;
Map<Integer, SeatStatus> seatStatusMap;
Map<Integer, ReentrantLock> seatLocks;
+ confirmSeats(List<Integer> seatIds)
+ releaseSeats(List<Integer> seatIds)
+ lockSeats(List<Integer> seatIds);
has-a
Movie
String name;
int length;
has-a
enum SeatStatus {
AVAILABLE,
LOCKED,
BOOKED
}
BookingController
BookingService bookService;
+ createBooking(User u, Show s, List<Integer> seats);
+ cancelBooking(Booking booking);
has-a
BookingService
Map<Integer, Booking> bookings;
// business logic for create and cancel booking
has-a
Booking
int bookingId;
User user;
Show show;
List<Integers> seats;
Payment payment;
has-a
has-a
has-a
Payment
int paymentId;
PaymentStatus status;
User
int userId;
String name;

## 16. Null Object Design Pattern (Behavioral Pattern)

### Null Object Pattern: (Behavior)

Let's Understanding the Problem:

Code without NULL checks leads to unexpected behavior due to NullPointerExceptions, resulting in a poor user experience. This is also a bad programming practice.

And that's one reason, we have to put so many '!=null' checks across the project to avoid NPE.

```
public interface Vehicle {
    public abstract void start();
    public abstract void stop();
}
```

Is-a

Is-a

```
public class Car extends Vehicle {
    //car specific fields
    private String color;
```

```
public class Bike extends Vehicle {
    //bike specific fields
```

```
public class Car extends Vehicle {
    //car specific fields
    private String color;
    private int seatingCapacity;

    public Car( String color, int seatingCapacity) {
        this.color = color;
        this.seatingCapacity = seatingCapacity;
    }

    @Override
    public void start() {
        System.out.println("Car is started and moving");
    }

    @Override
    public void stop() {
        System.out.println("Car is stopped");
    }

    public String getColor() {
        return color;
    }

    public int getSeatingCapacity() {
        return seatingCapacity;
    }
}
```

```
public class Bike extends Vehicle {
    //bike specific fields
    private String color;
    private int seatingCapacity;

    public Bike( String color, int seatingCapacity) {
        this.color = color;
        this.seatingCapacity = seatingCapacity;
    }

    @Override
    public void start() {
        System.out.println("Bike is started and moving");
    }

    @Override
    public void stop() {
        System.out.println("Bike is stopped");
    }

    public String getColor() {
        return color;
    }

    public int getSeatingCapacity() {
        return seatingCapacity;
    }
```

```java
public class VehicleFactory {

  public static Vehicle getVehicle (String type) {

    if (type.equals("car")) {
      return new Car( "Red", 5);
    }
    else if (type.equals("bike")) {
      return new Bike("Black", 2);
    }
    else {
      return null; // THE PROBLEM
    }
  }
}
```

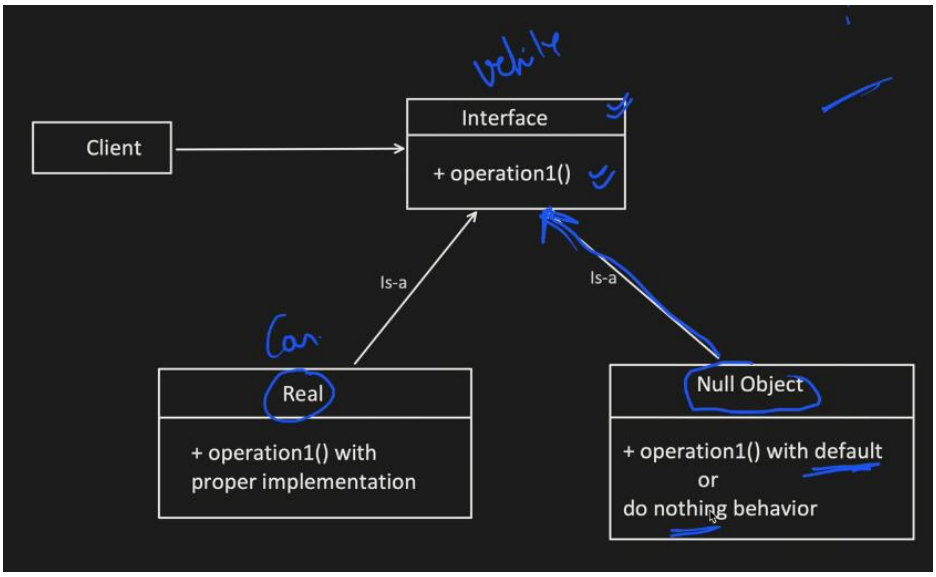
```java
public class ProblemDemo {
  public static void main(String[] args) {

    Vehicle truckObj = VehicleFactory.getVehicle("truck");
    if(truckObj  != null ) {    // THE PROBLEM
        truckObj .start();
    }
  }
}
```

*This !=null check is important to avoid NullPointerException*

## Solution: Null Object Pattern

- **The NULL Object Pattern is a behavioral design pattern that uses polymorphism to eliminate null checks.**
- **Instead of returning NULL and adding NULL checks wherever necessary, we return a special object called NULL OBJECT that implements the expected interface but does nothing (or provides default behavior).**
- This leads to clean code without redundant NULL checks everywhere.



```java
public interface Vehicle {
    public abstract void start();
    public abstract void stop();
}
```

Is-a

```java
blic class Car extends Vehicle {
```

```java
public class Bike extends Vehicle {
```

```java
public class NullVehicle extends Vehicle {
```

```java
public class NullVehicle extends Vehicle {
    //bike specific fields
    private String color = "Default";
    private int seatingCapacity = 0;

    public NullVehicle ( ) {

    }

    @Override
    public void start() {
        //do nothing
    }

    @Override
    public void stop() {
        //do nothing
    }

    public String getColor() {
        return color;
    }

    public int getSeatingCapacity() {
```

*Default behav*

udemy

```java
public class VehicleFactory {

    public static Vehicle getVehicle (String type) {

        if (type.equals("car")) {
            return new Car( "Red", 5);
        }
        else if (type.equals("bike")) {
            return new Bike("Black", 2);
        }
        else {
            return new NullVehicle(); // THE SOLUTION
        }
    }
}
```

null ✗

null be ok

```java
public class ProblemDemo {
    public static void main(String[] args) {

        Vehicle truckObj = VehicleFactory.getVehicle("truck");

        truckObj .start();

    }
}
```

*No need of !=null check now.*

!=null

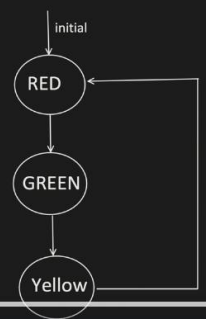
Benefits

- Makes code cleaner by eliminating repetitive null checks.
- Reduces NullPointerException risks.
- Increases code readability.

## 17. State Design Pattern (Behavioral) | Design Vending Machine

Let's understand this with an example of "Vending Machine" and "Traffic Signal"

1st : Traffic Signal (Simple)




| State | Operations |
|---|---|
| Red State (Stop) | action() -> Make Signal Red |
| Green State (Go) | action() ->Make Signal Green |
| Yellow State (Slow Down) | action() ->Make Signal Yellow |

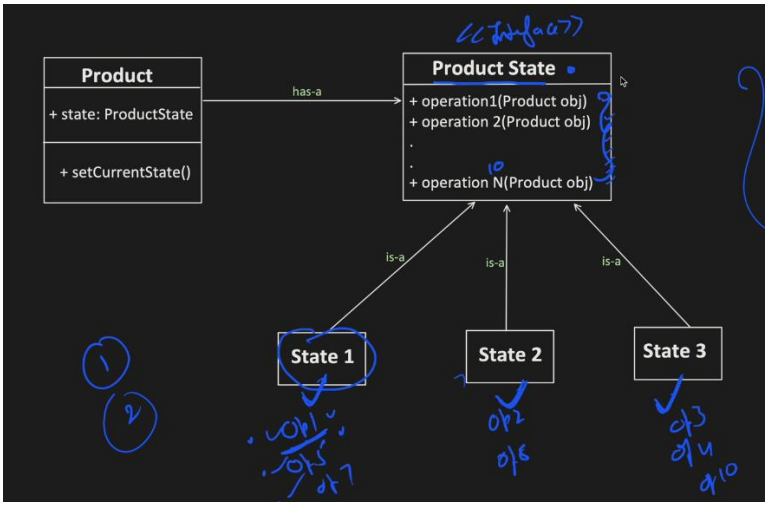
This type of problems, where Object change the state after performing certain operation can be solved through : State Design Pattern

Other Problem are:

- Design Television (state: On, Off)
- Design Vending Machine (state : Idle, Accept Coin, Product Selection etc.)

State Design Pattern:

**The State Pattern allows an object to change its behavior dynamically at runtime whenever there is a change in its internal state.**

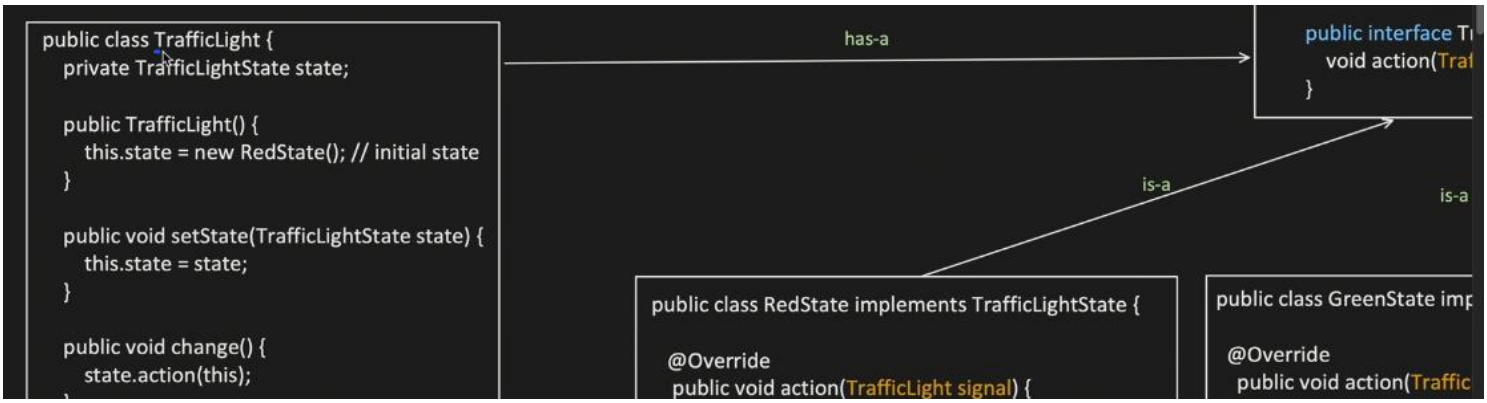


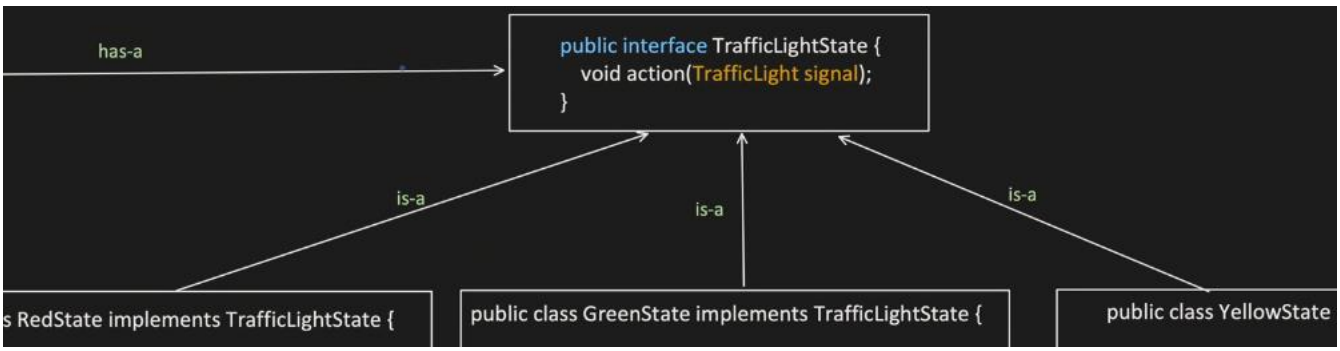
product state interface (interface or abstract class) - here internally we put all the operations which are possible within that particular state. We create multiple child classes for this product state.

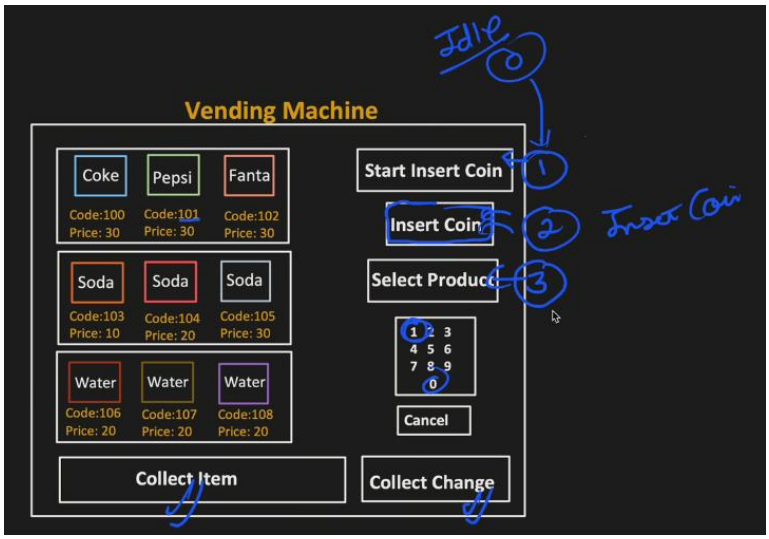
1) Each state implements provide their own logic for the particular operation which is eligible for that state.

So, we have all the operations possible in the interface or abstract class.
And each state provides an implementation for these operation(s) (implementation is provided for specific operations that are applicable for that state).
Each state provides an implementation for their only operation which they are only interested in.

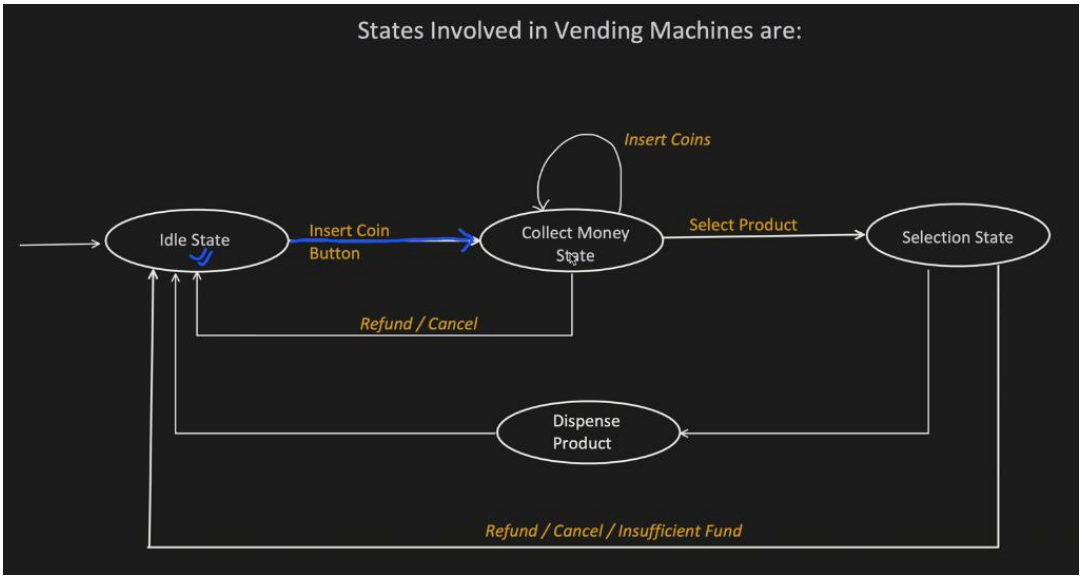
2) On each operation it will decide the next state. whether it must be in the same state after Performing this operation, or it must move to a different state, that is also what we can do.



States Involved in Vending Machines are:




| State | Operations |
|---|---|
| IdleState | • Press Insert Coin button |
| HasMoney State | • Insert Coin<br>• Select Product<br>• Cancel / Refund |
| Selection State | • Choose Product<br>• Return change<br>• Cancel/Refund full amount |
| Dispense Product State | • Dispense Product |

• Update Inventory

12h

30r

Inventory

ItemShelf[] inventory;

has-a

ItemShelf

int code;
Item item;
boolean soldOut;

Item

ItemType type;
int price;

```
public enum ItemType {
    COKE,
    PEPSI,
    JUICE,
    SODA;
}
```

```
public class VendingMachine {
    private State vendingMachineState;
    private Inventory inventory;
    private List<Coin> coinList;

    public VendingMachine(){
        vendingMachineState = new IdleState(); //initial IDLE state
        inventory = new Inventory(10); //no of items to be added
        coinList = new ArrayList<>();
    }

    public void setVendingMachineState(State vendingMachineState) {
        this.vendingMachineState = vendingMachineState;
    }
    public Inventory getInventory() {
        return inventory;
    }

    public void setInventory(Inventory inventory) {
        this.inventory = inventory;
    }
```

has-a

```java
public enum Coin {
    PENNY(1),
    NICKEL(5),
    DIME(10),
    QUARTER(25);

    public int value;

    Coin(int value) {
        this.value = value;
    }
}
```

Inventory

```java
ItemShelf[] inventory;
```

has-a

```java
public class VendingMachine {
    private State vendingMachineState;
    private Inventory inventory;
    private List<Coin> coinList;
```

has-a

```java
System.out.println("clicking on InsertCoinButton");
System.out.println("|");

//idle
State vendingState = vendingMachine.getVendingMachineState();
//clicking on InsertCoinButton
vendingState.clickOnInsertCoinButton(vendingMachine);

vendingState = vendingMachine.getVendingMachineState();
vendingState.insertCoin(vendingMachine, Coin.NICKEL);
vendingState.insertCoin(vendingMachine, Coin.QUARTER);
// vendingState.insertCoin(vendingMachine, Coin.NICKEL);

System.out.println("|");
System.out.println("clicking on ProductSelectionButton");
System.out.println("|");
vendingState.clickOnStartProductSelectionButton(vendingMachine);

vendingState = vendingMachine.getVendingMachineState();
vendingState.chooseProduct(vendingMachine, codeNumber: 102);
```

```
/Users/shrayanshjain/Library/Java/JavaVirtualMachines/ms-21.0
Currently Vending machine is in IdleState
|
filling up the inventory
|
CodeNumber: 101 Item: COKE Price: 12 isAvailable: true
CodeNumber: 102 Item: COKE Price: 12 isAvailable: true
CodeNumber: 103 Item: COKE Price: 12 isAvailable: true
CodeNumber: 104 Item: PEPSI Price: 9 isAvailable: true
CodeNumber: 105 Item: PEPSI Price: 9 isAvailable: true
CodeNumber: 106 Item: JUICE Price: 13 isAvailable: true
CodeNumber: 107 Item: JUICE Price: 13 isAvailable: true
CodeNumber: 108 Item: SODA Price: 7 isAvailable: true
CodeNumber: 109 Item: SODA Price: 7 isAvailable: true
CodeNumber: 110 Item: SODA Price: 7 isAvailable: true
|
clicking on InsertCoinButton
|
Currently Vending machine is in HasMoneyState
Accepted the coin
```

```
|
Currently Vending machine is in HasMoneyState
Accepted the coin
Accepted the coin
|
clicking on ProductSelectionButton
|
Currently Vending machine is in SelectionState
Returned the change in the Coin Dispense Tray: 18
Currently Vending machine is in DispenseState
Product has been dispensed
Currently Vending machine is in IdleState

Process finished with exit code 0
```

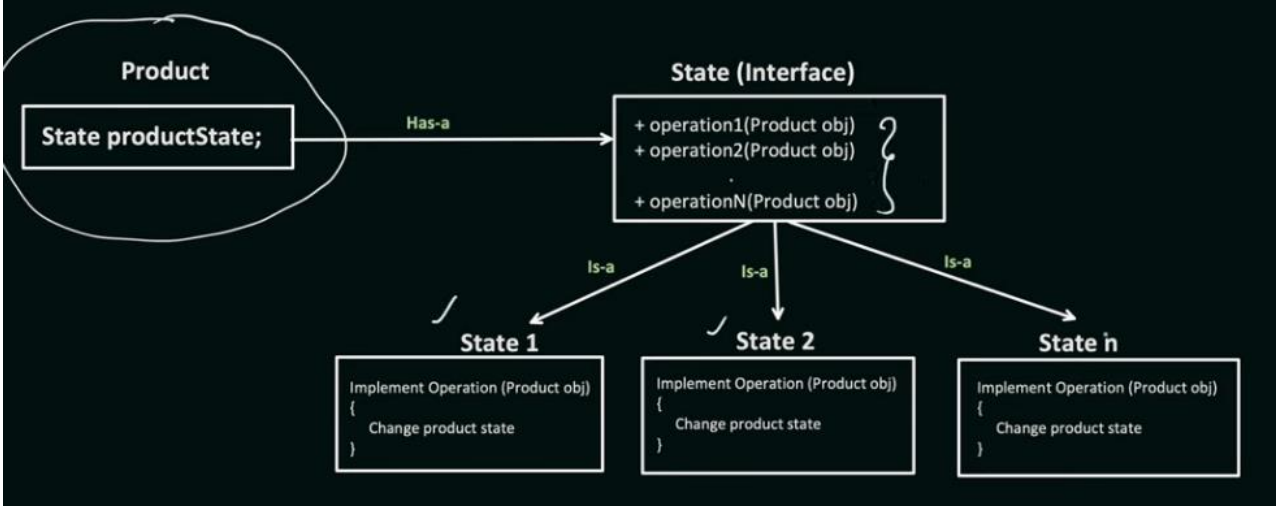
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/state?ref_type=heads

## Behavioral Design Patterns:

Guides how different objects communicate with each other effectively and Distribute tasks efficiently, making software system flexible and easy to maintain.

1. State Pattern
2. Observer Pattern
3. Strategy Pattern
4. Chain of Responsibility Pattern
5. Template Pattern
6. Interpreter Pattern
7. Command Pattern
8. Iterator Pattern
9. Visitor Pattern
10. Mediator Pattern
11. Memento Pattern

### 1. State Pattern: allows an object to alter its behaviour when its internal state changes.




```java
public class VendingMachine {

    VendingState machineState;

    public VendingState getMachineState() {
        return machineState;
    }

    public void setMachineState(VendingState machineState) {
        this.machineState = machineState;
    }
}
```

Has-a

```java
public interface VendingState {

    void insertCoin(VendingMachine product);

    void dispenseItem(VendingMachine product);
}
```

is-a

```java
public class IdleState implements VendingState{

    @Override
    public void insertCoin(VendingMachine product) {
        //insert coin logic
        System.out.println("Coin Inserted");
        product.setMachineState(new WorkingState());
    }

    @Override
    public void dispenseItem(VendingMachine product) {
        //not doing anything here
    }
}
```

is-a

```java
public class WorkingState implements VendingState{

    @Override
    public void insertCoin(VendingMachine product) {
        //not doing anything here
    }

    @Override
    public void dispenseItem(VendingMachine product) {
        System.out.println("Product dispensed");
        //set any other state if applicable
    }
}
```